TREC RAG 2026

Single-pass and Multi-stage Answer Generation

Nidhin Pattaniyil Tanwir Zaman Semih Yagli

npatta01@gmail.com zaman.tanwir@gmail.com semihyagli1@gmail.com

Independent researchers · Team NP Labs

Abstract

We compare two ways to answer TREC RAG requests using the same retrieved passages. Single-pass writes an answer directly. Multi-stage first plans an answer, then checks the draft for missing information and screens proposed edits. Multi-stage received higher organizer human-preference and coverage scores. On 25 topics, coverage of human-authored facts was 0.6578 versus 0.5705. We trace a submitted Green Bay topic from LLM-generated facets to evidence and both accepted answers. Three additional Retrieval baselines reuse the RAG pipeline's saved results.

1 Start with the Questions Inside the Request

TREC RAG supplies 119 long-form information requests, called *narratives*. A system searches the ClimbMix document collection and writes an answer of at most 1,024 words, citing its sources [2]. **Input:** a request and access to the collection. **Output:** an answer with document citations.

A request often asks several things at once. We use DeepSeek V4 Flash to split it into **facets**: focused questions that each cover part of the request. The model also writes search queries for those questions. We also search the original request. Each topic can have up to eight facets with 1–3 queries each: at most 25 searches in total.

Request: moving to Green Bay (rag2026-21)

> "How much of Green Bay's local identity, economy, and social life is really built around the Packers, and how much is just tourist branding?"

↓

DeepSeek extracts six facets and writes three queries for each

Economy Ownership Game-day culture

Local pride Rivalries Downsides

Ownership facet

"The unique community ownership structure of the Green Bay Packers and how it shapes local identity and fan involvement."

Search query

"Green Bay Packers community ownership"

Downsides facet

"Possible downsides or challenges of living in a city where one NFL team dominates local attention, economy, and social life."

Search query

"downsides living in Packers city"

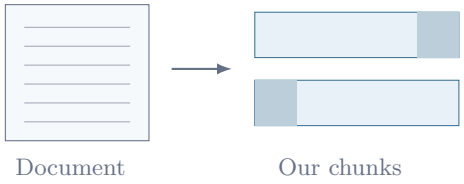
18 focused searches + the original full request = 19 searches

Figure 1: DeepSeek splits the Green Bay request into six facets. Two facets and their search queries are shown below the full set of headings.

2 Turn Retrieved Documents into Evidence and Nuggets

We create the passages ourselves from retrieved ClimbMix documents. Before either writer runs, we select exact source text and use Nuggetizer [3] to turn it into short, evidence-linked claims. These generated nuggets are advisory hints, not organizer gold nuggets.

1 Retrieve documents, then split their text locally



BM25 via Pyserini [6, 7]:
up to 1,000 documents per query.

Our chunks: up to 3,500 characters,
with 350 characters of overlap.

Mixedbread ranks query–passage pairs [8];
keep up to 100 passages per query.

2 Keep useful evidence without repeating it

Group repeated evidence. Match identical text first, then use MiniLM embeddings [9] to find similar text. Check type, numbers, dates, names, and negation before grouping.

Keep the source wording. Retain the strongest representative and its source links. Select up to 40 groups per facet, balancing relevance against repetition [10].

Selected evidence for the downsides facet

"For those who live near football stadiums, the noise can be a constant source of frustration and disturbance."

Exact source sentence; citation link retained.

3 Generate a separate nugget set for each facet

Each facet separately: its selected evidence → one Nuggetizer call
→ up to **20 nuggets for that facet**. Each nugget links to its evidence and at most three supporting documents.

Generated nugget for the same facet

"Noise from NFL stadiums can be a constant source of frustration and disturbance for nearby residents."

A model-written hint; the source evidence remains authoritative.

Freeze one shared input for both writers

Original request + exact evidence + generated nuggets + allowed citation IDs

Figure 2: From retrieved documents to shared writer input: split the text, select evidence, then generate nuggets for each facet. The Green Bay example shows a source sentence and its linked nugget.

Nuggets help organize the answer; writers must ground their claims in the original evidence. Both writers receive the same request, evidence, and nuggets. Neither searches again or reads organizer judgments. This decomposition-and-selection approach builds on earlier TREC systems [4, 5].

3 Write Two Answers from the Same Evidence

GPT-5.6 Sol writes both answers. Single-pass writes directly from the shared evidence. Multi-stage adds GPT-5.6 Luna to plan the answer, look for omissions, and screen proposed edits.

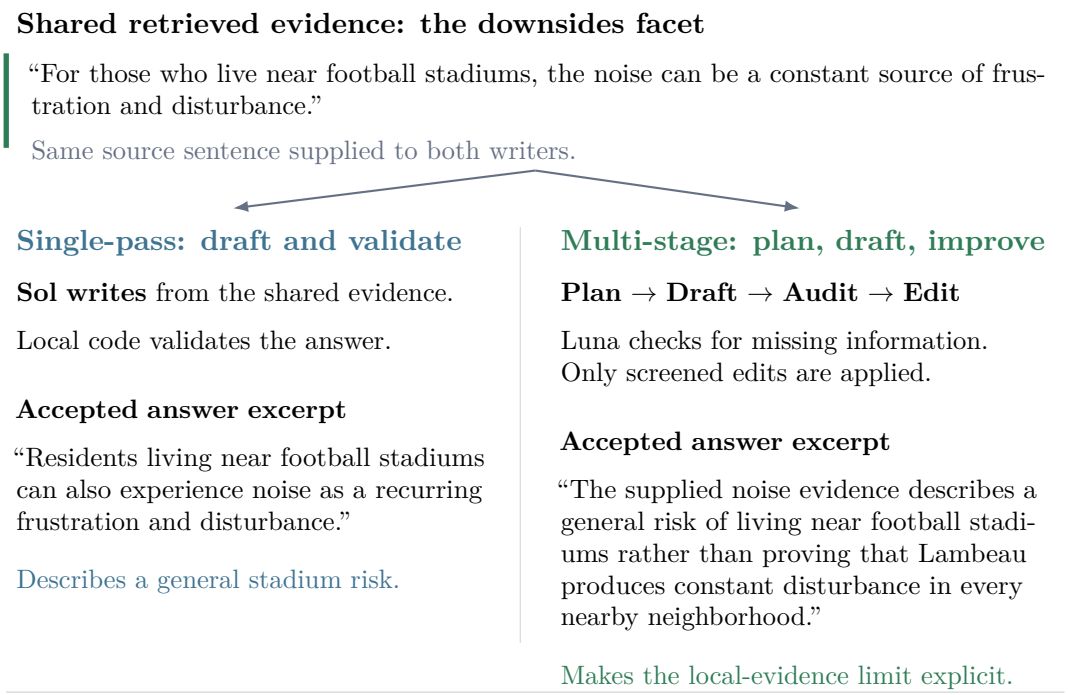


Figure 3: Answer excerpts for the Green Bay topic. Both cite the same source; multi-stage explicitly distinguishes a general stadium risk from evidence about Lambeau.

Sol proposes edits; Luna screens them for support, usefulness, repetition, and lost qualifications. Local code applies accepted edits and checks citations and length. Multi-stage keeps the validated draft if revision fails. Both methods produced 119 answers that passed these format checks.

4 Organizer RAG Results

Multi-stage scored higher on both human preference and information coverage. The organizer evaluated subsets of the 119 submitted topics [1]. Reference medians weight runs equally, rather than teams; they do not give our leaderboard rank.

Human preference. Human assessors compared answers in two disjoint batches: 17 topics with narrative-based rubrics, and 25 topics with expected facts gathered from the corpus. The organizer fitted Bradley–Terry preference ratings separately for each batch. Multi-stage was above the run median in both; single-pass was below it in Batch 1 and near it in Batch 2.

Batch	Single-pass [95% CI]	Multi-stage [95% CI]	Run median
1 (17 topics)	1019.7 [970.5, 1068.9]	1179.8 [1128.6, 1231.0]	1048.7
2 (25 topics)	1026.0 [983.8, 1068.3]	1181.4 [1134.6, 1228.2]	1024.8

Table 1: Organizer human-preference ratings. Higher is better. Each batch has its own fitted Bradley–Terry scale; compare systems within a row.

The 95% intervals do not overlap within either batch. They use a judgment-level sandwich estimator; we performed no additional paired significance test.

Information coverage. Coverage was judged by GPT-6.1 Sol at medium reasoning. *Vital strict* counts the proportion of essential items fully supported by an answer; *all strict* includes all items equally. Partial support earns no credit. The three collections are reported separately in Table 2.

Collection	Measure	Single-pass	Multi-stage	Run median
Batch 1 rubrics (17)	Vital strict	0.8431	0.8804	0.7157
	All strict	0.7740	0.8738	0.7325
Batch 2 human nuggets (25)	Vital strict	0.5383	0.6338	0.4879
	All strict	0.5705	0.6578	0.5556
Batch 2 automatic nuggets (25)	Vital strict	0.1647	0.3147	0.1547
	All strict	0.1280	0.2496	0.1280

Table 2: Organizer RAG coverage scores, averaged equally across topics within each collection. Higher is better. Medians summarize evaluated runs, not teams. The collections contain different items and should not be compared as if they were one test.

Multi-stage scored higher on all six coverage measures. On human-authored Batch-2 nuggets, the gain over single-pass was 8.73 percentage points (0.6578 versus 0.5705). Both exceeded the run median for that measure.

5 Derived Retrieval Baselines

The Retrieval task asks for a ranked document list. Our three baselines reuse the RAG pipeline's saved searches and passage scores. Within each narrative or facet pool, we keep documents whose scores stand well above the typical score, using a cutoff adjusted for the spread of scores. We combine these selections into one document set. Each run contains the same 4,246 topic–document pairs, returning 1–121 documents per topic.

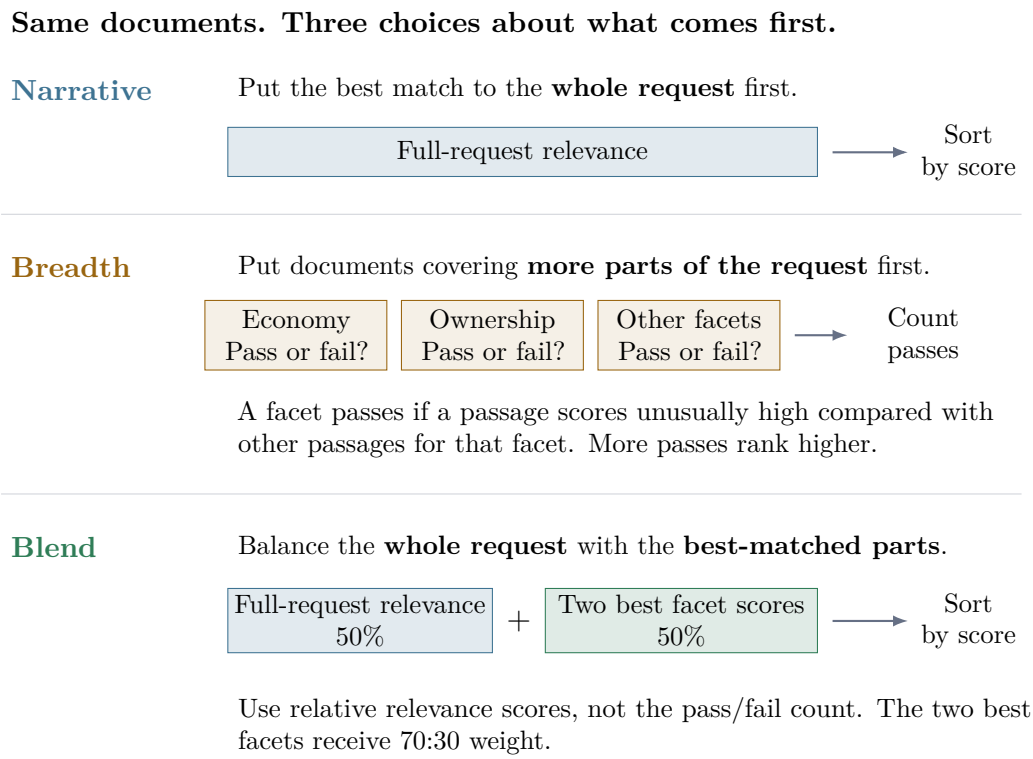


Figure 4: The three derived Retrieval baselines change the ordering, not the document set. Narrative and Blend use percentile relevance scores. Breadth counts facets passing a raw-score threshold, breaking ties by strong-passage count and then Blend.

On 82 topics, automated UMBRELA judgments [11] treated relevance grades of at least 2 as relevant. Precision measures the relevant share of returned documents; recall measures the share of judged-relevant documents returned. We had higher precision but lower recall than the run median (Table 3).

Measure	Topics	All three runs	Run median
Document precision	82	0.4116	0.3499
Document recall	82	0.0897	0.1002
Mean submitted documents	82	38.6	100.0
Gain: human-authored nuggets	25	0.7839	0.8965
Gain: automatic nuggets	25	0.4033	0.6268
Mean submitted documents	25	31.8	100.0

Table 3: Organizer Retrieval results. Medians cover 114 evaluated runs from 27 teams, weighting each run equally. Scores and document counts are topic means; the two count rows use different topic subsets. Higher precision, recall, and gain are better; document count describes effort.

On 25 topics, the organizer also measured coverage of expected facts, or nuggets. Each fully supported nugget counts once, weighted 1 for essential facts and 0.5 for other useful facts. Assignments were automated for both human-authored and automatic nuggets. Both gain scores were below the run medians.

All three rankings received identical scores, including per-topic scores: the evaluation measures the full document set, which the rankings share. These results assess our selection, not which ordering puts useful documents first.

6 What the RAG Comparison Establishes

With the same evidence, multi-stage received higher human-preference and coverage scores. It also wrote longer answers: 531–1,011 words versus 203–749. Planning, auditing, revision, model mix, and length all changed together. We cannot attribute the improvement to one stage.

A separate retrospective citation-support check favored single-pass; the organizer gains should not be read as better citation grounding.

The organizer results favor the complete multi-stage configuration under fixed evidence. They do not isolate which stage caused the gain or whether answer length explains it. Retrieval coverage and citation support remain separate evaluation dimensions.

Implementation and assistance. The RAG runs were `rag26-ss1` (Single-pass) and `rag26-ms1-final` (Multi-stage, priority 1). Blend (`r26-narr-facet-v1`) was the prioritized Retrieval baseline. Accepted files remain unchanged. Codex assisted development, analysis, and drafting; author review is pending.

References

[1] TREC RAG Track Organizers. *Retrieval and RAG task results*. Participant-specific TSV score releases and accompanying evaluation READMEs, received November 2026; inspected November 7, 2026. Includes overall run reference statistics. Unpublished participant communication.

[2] TREC RAG organizers. *TREC RAG 2026: Retrieval-Augmented Generation Track*. 2026. `https://trec-rag.github.io/`. Accessed November 7, 2026.

[3] Ronak Pradeep, Nandan Thakur, Shivani Upadhyay, Daniel Campos, Nick Craswell, and Jimmy Lin. *The Great Nugget Recall: Automating Fact Extraction and RAG Evaluation with Large Language Models*. In Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval, pages 180–190, 2025. Software: `https://github.com/castorini/nuggetizer`.

[4] Yukio Uematsu, Koyuki Otani, Tomoyuki Shiroyama, and Masaaki Tsuchida. *Keystone-Docs RAG at TREC 2025: Coverage-Aware Few-Document Augmentation via Narrative Decomposition*. In Proceedings of TREC 2025. `https://trec.nist.gov/pubs/trec34/papers/tus.rag.pdf`.

[5] Zizhen Li and Hai-Tao Yu. *Submodular Evidence Selection for Grounded Answer Generation in TREC RAG 2025*. In Proceedings of TREC 2025. `https://trec.nist.gov/pubs/trec34/papers/WING-II.rag.pdf`.

[6] Stephen Robertson and Hugo Zaragoza. *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 3(4):333–389, 2009. `https://doi.org/10.1561/1500000019`.

[7] Jimmy Lin, Xueguang Ma, Sheng-Chieh Lin, Jheng-Hong Yang, Ronak Pradeep, and Rodrigo Nogueira. *Pyserini: A Python Toolkit for Reproducible Information Retrieval Research with Sparse and Dense Representations*. SIGIR, pages 2356–2362, 2021. `https://github.com/castorini/pyserini`.

[8] Xianming Li, Aamir Shakir, Rui Huang, Julius Lipp, Benjamin Clavié, and Jing Li. *ProRank: Prompt Warmup via Reinforcement Learning for Small Language Models Reranking*. arXiv:2506.03487, 2025. Model used: `mxbai-rerank-base-v2`. `https://arxiv.org/abs/2506.03487`.

[9] Sentence Transformers. *all-MiniLM-L6-v2*. Model card. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2. Accessed November 7, 2026.

[10] Jaime Carbonell and Jade Goldstein. *The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries*. SIGIR, pages 335–336, 1998. https://doi.org/10.1145/290941.291025.

[11] Shivani Upadhyay, Ronak Pradeep, Nandan Thakur, Nick Craswell, and Jimmy Lin. *UMBRELA: UMbrela is the (Open-Source Reproduction of the) Bing RELevance Assessor*. arXiv:2406.06519, 2024. https://arxiv.org/abs/2406.06519.